

RISING WATERS

Project Description:

Rising Waters is a machine learning-powered flood prediction system designed to provide early warning capabilities to disaster management authorities and meteorologists. The platform analyses environmental and meteorological data using four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost — to predict the likelihood of a flood event in a given region.

The system is built using Python and Flask, trained on a real-world dataset of over one million records sourced from Kaggle, containing twenty environmental features such as monsoon intensity, deforestation levels, urbanization scores, coastal vulnerability, and drainage system quality. The best performing model, XGBoost with an accuracy of 81.60%, is saved and integrated into a Flask web application that provides an intuitive interface for real-time flood risk assessment

The platform includes three core components. The Flood Risk Predictor accepts twenty environmental parameter values and returns an instant flood risk classification of either Flood Likely or No Flood. The Risk Level Indicator visually displays the severity of the predicted risk using a color coded progress bar ranging from low to high. The Recommendations Engine provides actionable guidance tailored to the prediction result, advising authorities on evacuation, resource deployment, or continued monitoring as appropriate.

Scenarios

Scenario 1: Early Flood Warning and Evacuation Planning

A meteorologist enters current monsoon intensity, drainage system quality, and coastal vulnerability readings for a flood prone district during heavy rainfall season. The Rising Waters system analyses the twenty input parameters and predicts a high probability of flooding, displaying a Flood Likely result in red. Authorities use this prediction to issue evacuation advisories several hours in advance, potentially saving thousands of lives.

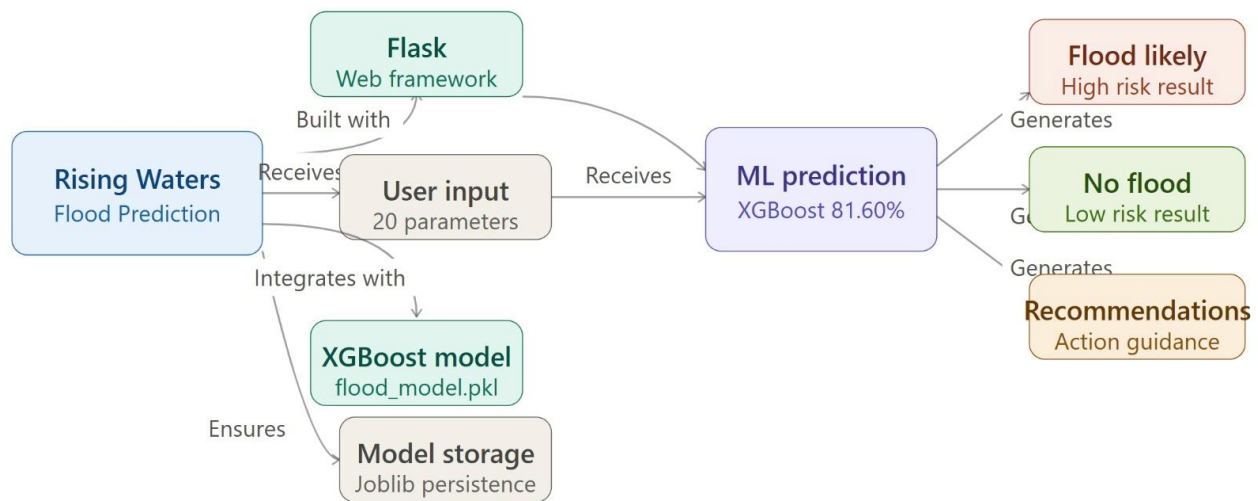
Scenario 2: Disaster Response and Resource Allocation

A disaster relief coordinator uses the web application during monsoon season to monitor multiple regions simultaneously. By entering regional weather and environmental data for each area, the system provides instant flood risk classifications. Regions predicted as high risk receive priority deployment of rescue teams, medical camps, and relief materials, while low risk regions are placed on standby monitoring.

Scenario 3: Model Validation and Performance Assessment

A government data analyst tests the Rising Waters system against a historical flood event dataset to evaluate its reliability. The XGBoost model achieves 81.60% accuracy on the test data, outperforming Decision Tree at 64.03%, KNN at 73.23%, and Random Forest at 77.63%. The analyst confirms the system is sufficiently reliable for operational use in regional flood risk monitoring programs.

Technical Architecture:



Description: Rising Waters utilises a modular three-tier architecture to deliver fast, machine learning-driven flood risk predictions. The frontend provides a responsive user interface built with HTML and CSS, while the Flask-based backend handles form validation, data processing, and model inference. The trained XGBoost model, saved as flood_model.pkl using joblib, is loaded at application startup and used to classify each prediction request as either Flood Likely or No Flood. The result is returned to the user along with a risk level indicator and a set of actionable recommendations tailored to the prediction outcome. The application is designed for local deployment via Flask and can be hosted publicly on Railway for remote access.

Pre-requisites:

- **Flask Framework Knowledge** — Flask Documentation: <https://flask.palletsprojects.com>
- **Machine Learning with Scikit-learn** — Scikit-learn Documentation: <https://scikit-learn.org>
- **XGBoost Library** — XGBoost Documentation: <https://xgboost.readthedocs.io>
- **HTML, CSS and JavaScript Skills** — W3Schools Tutorials: <https://www.w3schools.com>
- **Python Programming Proficiency** — Python Documentation: <https://www.python.org/doc>
- **Data Handling with Pandas and NumPy** — Pandas Documentation: <https://pandas.pydata.org>
- **Version Control with Git** — Git Documentation: <https://git-scm.com/doc>
- **Development Environment Setup** — VS Code Download: <https://code.visualstudio.com>
- **Railway for Cloud Deployment** — Railway Documentation: <https://docs.railway.app>

Project Workflow:

The development of Rising Waters followed a structured workflow divided into six major milestones, progressing from dataset collection through to final deployment.

Milestone 1: Dataset Selection and Environment Setup

- **Activity 1.1: Obtain the Dataset**
- The flood prediction dataset was sourced from Kaggle, containing over 1.1 million records with 20 environmental and meteorological features. The train.csv and test.csv files were downloaded and placed inside the dataset folder of the project.
- **Activity 1.2: Set Up the Development Environment**
- Python 3.10, VS Code, and required libraries (pandas, numpy, scikit-learn, xgboost, flask, joblib) were installed. A structured project folder was created containing separate directories for the dataset, the trained model, and HTML templates.
- **Activity 1.3: Define the Application Architecture**
- The five-layer architecture was planned — User Layer, Presentation Layer, Application Layer, Machine Learning Layer, and Data Layer — outlining how user input flows through the Flask backend to the trained model and back to the result page.

Milestone 2: Data Preprocessing and Model Training

- **Activity 2.1: Load and Explore the Dataset**
- The dataset was loaded using pandas and explored to check shape, missing values, and statistical summaries. No missing values were found across all 22 columns.
- **Activity 2.2: Preprocess the Data**
- The id column was removed, the continuous Flood Probability column was converted into a binary Flood label using a 0.5 threshold, and the data was split into features and target. An 80/20 train-test split was applied on a 50,000 row sample for efficient training.

Milestone 3: Model Building and Evaluation

- **Activity 3.1: Train Multiple Classifiers**
- Four models — Decision Tree, Random Forest, KNN, and XGBoost — were trained on the preprocessed training data.
- **Activity 3.2 — Compare and Save Best Model**
- Accuracy scores were compared across all four models. XGBoost achieved the highest accuracy of 81.60% and was saved as flood_model.pkl using joblib for use in the web application.

Milestone 4: Flask Web Application Development

- **Activity 4.1: Build app.py**
- The Flask backend was developed with three routes — home (/), prediction (/predict), and about (/about) — handling form submission, model loading, and result rendering.
- **Activity 4.2 — Connect Model to Routes**

- The saved XGBoost model was loaded once at application startup and used to generate real-time flood risk predictions based on form input.

Milestone 5: Frontend Development

- **Activity 5.1: Design the User Interface**
- Three HTML pages were created — index.html for the input form, result.html for displaying predictions, and about.html for project information — styled with a dark glassmorphism theme.
- **Activity 5.2: Add Interactive Elements**
- A risk level indicator bar, animated loading overlay, color-coded result banners, and recommendation sections were added to enhance usability.

Milestone 6: Testing and Deployment

- **Activity 6.1 — Performance Testing**
- Load and stress testing was conducted using Locust with 10 to 100 virtual users, confirming zero failures and response times well within acceptable limits.
- **Activity 6.2 — Prepare for Deployment**
- requirements.txt, Procfile, and runtime.txt were created to support cloud deployment.
- **Activity 6.3 — Deploy on Railway**
- The project will be uploaded to GitHub and deployed on Railway to generate a publicly accessible live URL.

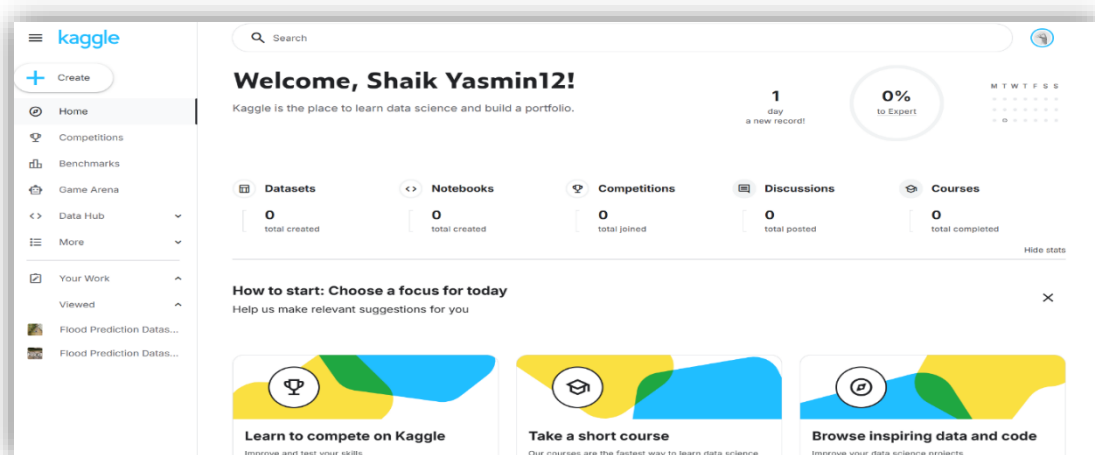
Milestone 1: Dataset Selection and Environment Setup

In this milestone, the focus is on selecting and preparing the flood prediction dataset and setting up the development environment needed to build the Rising Waters application. This involves downloading a real-world dataset from Kaggle, configuring Python and VS Code, and installing all required libraries for data science and web development.

Activity 1.1: Obtain the Dataset from Kaggle

Before training can begin, you need a flood prediction dataset. Follow the steps below to download it.

- **Step 1-Create a Kaggle Account:** Visit kaggle.com and sign up using your Google account or email.



- flood prediction dataset**

<> Notebooks 1,300 Topics 148 Comments 101 Datasets 57 Competitions 2 Hugging Face Models 2

Filter by Relevance ▾

DATE

 - Last 90 days 140
 - This week 5

VIEWS BY YOU

 - Viewed 2
 - Not Viewed 1,608

CREATOR

 - You 0
 - Others 1,608

TOPIC TYPE

 - Writeups 20
 - All Topics and Writeups 148

1,610 Results

Flood Prediction Dataset
Dataset - 7mo ago · by Naiya Khalid
Regression with [flood prediction](#) on full [dataset](#)

91
12,494 downloads

Flood probability Predictions and EDA
Notebook - 2y ago · by Monish Kumar D
filename in filenames: print(os.path.join(dirname, filename)) df = pd.read_csv("../kaggle/input/flood-prediction-dataset")

+ 2

Comment on:
Discussion Comment - 2y ago · by Santhosh T
In the Flood Prediction Dataset forum

- 0
0 replies

Flood Prediction Dataset
Notebook - 10mo ago · by Saja Saad
kaggle/hemp/, but they won't be saved outside of the current session df = pd.read_csv("../kaggle/input/flood-predicti")

- 6
0 comments

Regression Flood Analysis
Notebook - 2y ago · by Satya
****Dataset**:** The dataset is derived from the [Flood Prediction Factors dataset](#).

- 283
38 comments

- | Name | Type | Compressed size | Password pr... | Size | Ratio | Date modified |
|---|--------------------------------|-----------------|----------------|-----------|-------|------------------|
|  flood.csv | Microsoft Excel Comma Separ... | 680 KB | No | 2,300 KB | 71% | 07-12-2025 20:19 |
|  sample_submission.csv | Microsoft Excel Comma Separ... | 1,714 KB | No | 8,735 KB | 81% | 07-12-2025 20:19 |
|  test.csv | Microsoft Excel Comma Separ... | 10,093 KB | No | 35,253 KB | 72% | 07-12-2025 20:19 |
|  train.csv | Microsoft Excel Comma Separ... | 16,819 KB | No | 57,742 KB | 71% | 07-12-2025 20:19 |

- **Step 1** — Install Python and verify with `python -version`

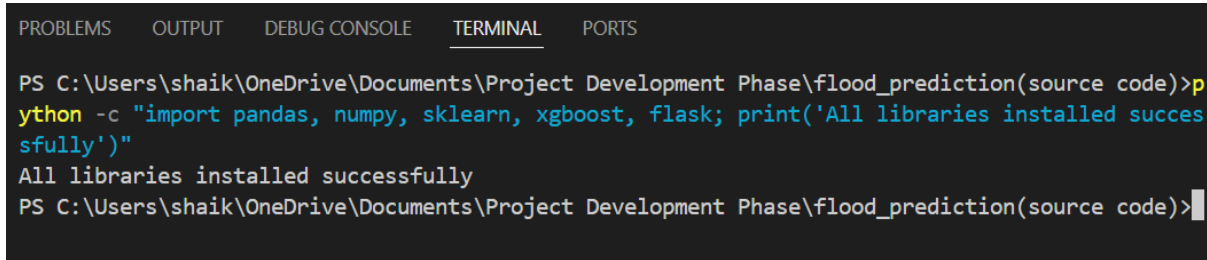
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\shaik\OneDrive\Documents\Project Development Phase\flood_prediction(source code)> python --version

Python 3.14.6
PS C:\Users\shaik\OneDrive\Documents\Project Development Phase\flood_prediction(source code)>

```

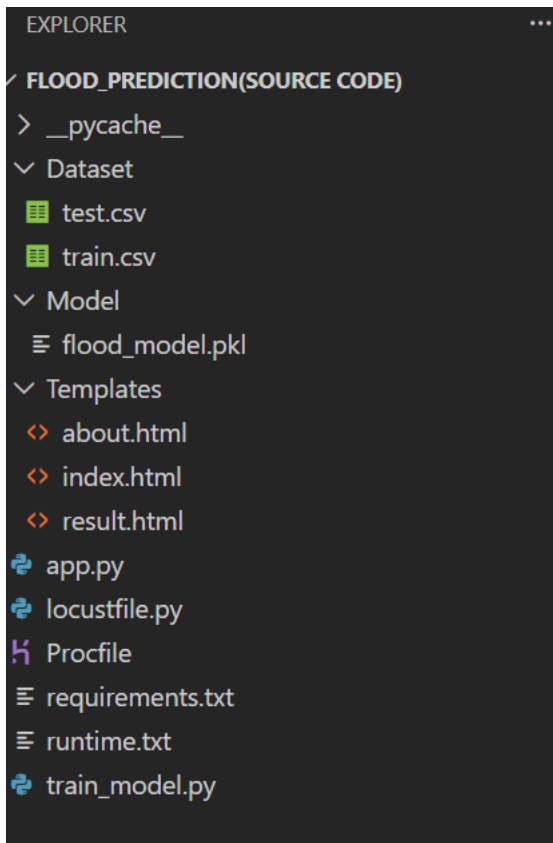
- **Step 2** — Install VS Code and the Python extension
- **Step 3** — Install required libraries using pip



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shaik\OneDrive\Documents\Project Development Phase\flood_prediction(source code)>python -c "import pandas, numpy, sklearn, xgboost, flask; print('All libraries installed successfully')"
All libraries installed successfully
PS C:\Users\shaik\OneDrive\Documents\Project Development Phase\flood_prediction(source code)>
```

- **Step 4** — Create the project folder structure



```
EXPLORER

✓ FLOOD_PREDICTION(SOURCE CODE)
  > __pycache__
  ✓ Dataset
    test.csv
    train.csv
  ✓ Model
    flood_model.pkl
  ✓ Templates
    about.html
    index.html
    result.html
  app.py
  locustfile.py
  Procfile
  requirements.txt
  runtime.txt
  train_model.py
```

Milestone 2: Data Preprocessing and Model Training

This milestone covers loading the raw dataset, cleaning it, and preparing it for machine learning by converting the target column into a binary classification label.

Activity 2.1: Load and Explore the Dataset

The train_model.py script loads the dataset using pandas and prints the shape, first five rows, missing value counts, and statistical summary to verify data quality.

```
PS C:\Users\shaik\OneDrive\Documents\Project Development Phase\flood_prediction(source code)>python train_model.py
Loading dataset...
Full Dataset Shape: (1117957, 22)
Using Sample of: 50000 rows

Preprocessing data...
Training set: 40000 rows
Testing set: 10000 rows
```

Activity 2.2: Preprocess the Dataset

The id column is dropped, FloodProbability is converted into a binary Flood column using a 0.5 threshold, and the dataset is split into training and testing sets using an 80/20 ratio.

```
Training set: 40000 rows
Testing set: 10000 rows
```

Milestone 3: Model Building and Evaluation

This milestone focuses on training multiple machine learning classifiers, comparing their performance, and saving the best one for deployment.

Activity 3.1: Train Multiple Classifiers

Four models — Decision Tree, Random Forest, KNN, and XGBoost — are trained one after another on the same training data, with accuracy recorded for each.

```
--- Training Models ---

Training Decision Tree...
Decision Tree Accuracy: 64.03%
Time: 0.45 seconds

Training Random Forest...
Random Forest Accuracy: 77.63%
Time: 3.10 seconds

Training KNN...
KNN Accuracy: 73.23%
Time: 6.48 seconds

Training XGBoost...
XGBoost Accuracy: 81.60%
Time: 0.39 seconds
```

Activity 3.2: Compare and Save the Best Model

After training, the accuracy scores are compared in a summary table and the best performing model, XGBoost at 81.60% accuracy, is saved as flood_model.pkl.

```
--- Model Comparison ---
Decision Tree: 64.03%
Random Forest: 77.63%
KNN: 73.23%
XGBoost: 81.60%

Best Model: XGBoost
Best model saved to model/flood_model.pkl

All done!
```

Milestone 4: Flask Web Application Development

This milestone establishes the backbone of the Rising Waters application by writing the Flask routes that connect the trained model to a usable web interface.

Activity 4.1: Define Core Routes in app.py

Three routes are created — the home route (/) renders the input form, the about route (/about) renders the project information page, and the predict route (/predict) accepts form data, runs it through the model, and returns a prediction.

```
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/predict', methods=['POST'])
def predict():
```

Activity 4.2: Run and Test the Flask Server Locally

The application is started using python app.py and tested by opening it in a browser at <http://127.0.0.1:5000>.

```
PS C:\Users\shaik\OneDrive\Documents\Project Development Phase\flood_prediction(source code)>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 862-910-255
```

Milestone 5: Frontend Development

This milestone focuses on designing a clean, professional, and responsive user interface for entering data and viewing flood predictions.

Activity 5.1: Build the Home Page (index.html)

A form is created with 20 input fields grouped into five categories — Rainfall and Climate Factors, Infrastructure and Management, Environmental Factors, Social and Urban Factors, and Risk and Vulnerability Factors — styled with a dark glassmorphism theme.

 **FloodGuard AI**

[Home](#) [About](#)

 **Flood Prediction System**

Enter environmental and meteorological data below to get an instant AI-powered flood risk assessment

81.6%

Model Accuracy

4

ML Algorithms

1M+

Records Trained

20

Features Analyzed

 **Enter Environmental Data**

 **RAINFALL & CLIMATE FACTORS**

 **Monsoon Intensity**

Enter 1-10 

1 = Very Low, 10 = Extremely High

 **Topography Drainage**

Enter 1-10 

1 = Good Drainage, 10 = Poor Drainage

 **Climate Change**

Enter 1-10 

1 = Minimal Impact, 10 = Severe Impact

 **Siltation**

Enter 1-10 

1 = Low Silt, 10 = High Silt

 **INFRASTRUCTURE & MANAGEMENT**

 **River Management**

Enter 1-10 

1 = Well Managed, 10 = Poorly Managed

 **Drainage Systems**

Enter 1-10 

1 = Excellent, 10 = Very Poor

 **Dams Quality**

Enter 1-10 

1 = Excellent, 10 = Very Poor

 **Deteriorating Infrastructure**

Enter 1-10 

1 = Good Condition, 10 = Very Poor

 **ENVIRONMENTAL FACTORS**

 **Deforestation**

Enter 1-10 

1 = Low, 10 = Extreme

 **Watersheds**

Enter 1-10 

1 = Healthy, 10 = Degraded

 **Wetland Loss**

Enter 1-10 

1 = Minimal Loss, 10 = Severe Loss

 **Agricultural Practices**

Enter 1-10 

1 = Sustainable, 10 = Harmful

 **SOCIAL & URBAN FACTORS**

 **Urbanization**

Enter 1-10 

1 = Low, 10 = Extreme

 **Population Score**

Enter 1-10 

1 = Sparse, 10 = Overcrowded

 **Encroachments**

Enter 1-10 

1 = Minimal, 10 = Severe

 **Political Factors**

Enter 1-10 

1 = Stable, 10 = Unstable

 **RISK & VULNERABILITY FACTORS**

 **Ineffective Disaster Preparedness**

Enter 1-10 

1 = Well Prepared, 10 = Unprepared

 **Landslides**

Enter 1-10 

1 = Low Risk, 10 = Extreme Risk

 **Coastal Vulnerability**

Enter 1-10 

1 = Low Risk, 10 = Extreme Risk

 **Inadequate Planning**

Enter 1-10 

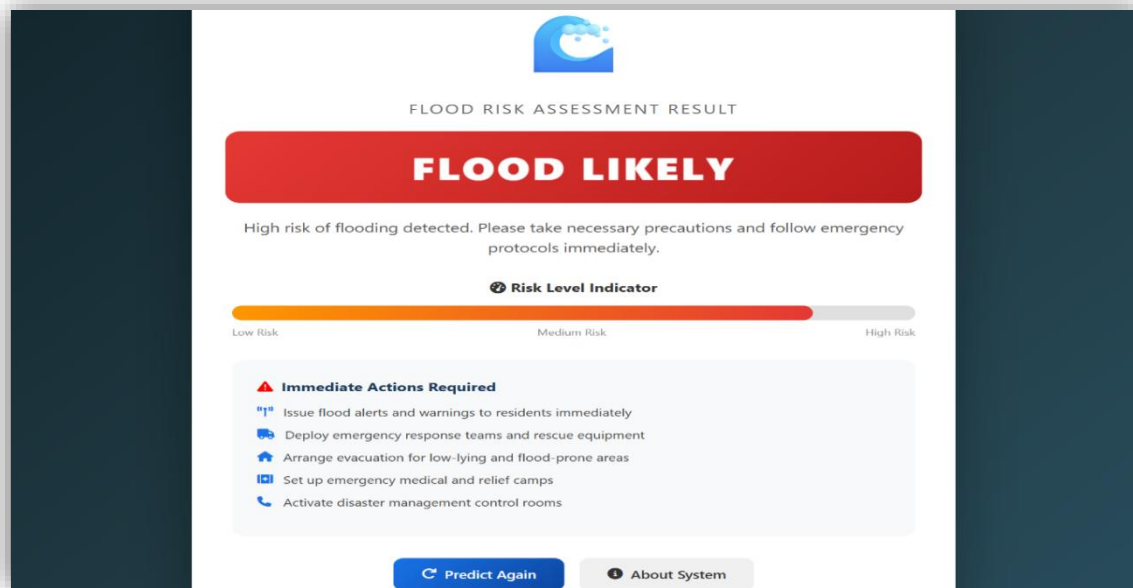
1 = Well Planned, 10 = Poorly Planned

 **Analyze Flood Risk Now**

 FloodGuard AI — Powered by XGBoost Machine Learning | Built with Flask

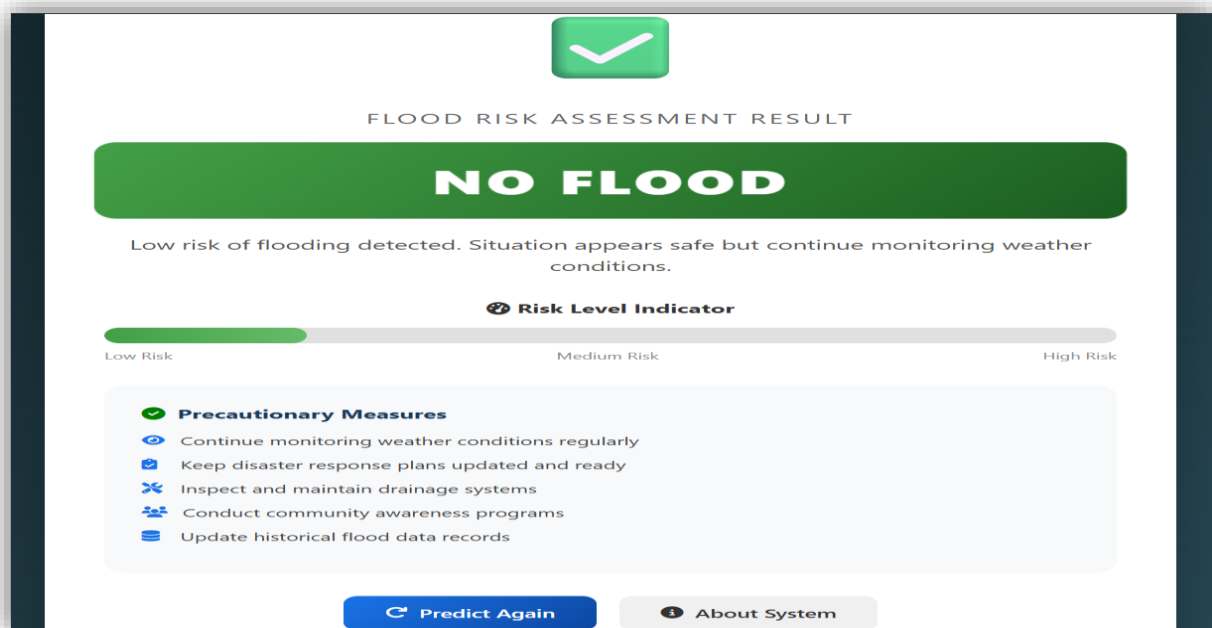
Activity 5.2: Build the Result Page (result.html)

The result page displays the prediction in a color-coded banner (red for Flood Likely, green for No Flood), along with a risk level indicator bar and tailored recommendations.



Activity 5.3: Build the About Page (about.html)

The about page summarizes the project, technology stack, and model accuracy comparison table for evaluators and users to review.



Milestone 6: Testing and Deployment

This final milestone verifies that the application performs reliably under load and prepares it for public deployment.

Activity 6.1: Performance Testing with Locust

Load testing is performed using Locust with 10 concurrent virtual users to measure response time, throughput, and error rate.


FloodGuard AI
Home
About

About FloodGuard AI

A machine learning powered flood prediction and early warning system


Project Objective

To build an accurate and accessible flood prediction system using machine learning that helps authorities issue early warnings and manage disaster response effectively.


Dataset

Trained on over 1 million records with 20 environmental and meteorological features including monsoon intensity, deforestation levels, urbanization scores and more.


Technology Stack

Python, Scikit-learn, XGBoost, Flask, HTML, CSS. The best performing model is saved and served through a Flask web application.


Deployment

Designed for deployment on IBM Cloud making it scalable and accessible from any location by disaster management authorities.

Model Performance Comparison

Model	Accuracy	Status
Decision Tree	64.03%	Baseline
K-Nearest Neighbours	73.23%	Good
Random Forest	77.63%	Better
XGBoost	81.60%	 Best Model (Deployed)

How It Works

1. User enters 20 environmental and meteorological values into the form
2. The Flask app receives the input and passes it to the XGBoost model
3. The model analyses the data based on patterns learned from 1 million+ records
4. A prediction is returned — either FLOOD LIKELY or NO FLOOD
5. The result page displays the risk level along with recommended actions

← Back to Prediction


 FloodGuard AI — Powered by XGBoost Machine Learning | Built with Flask

 LOCUST

Host

http://127.0.0.1:5000

Status

RUNNING

Users

10

RPS

4.94

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

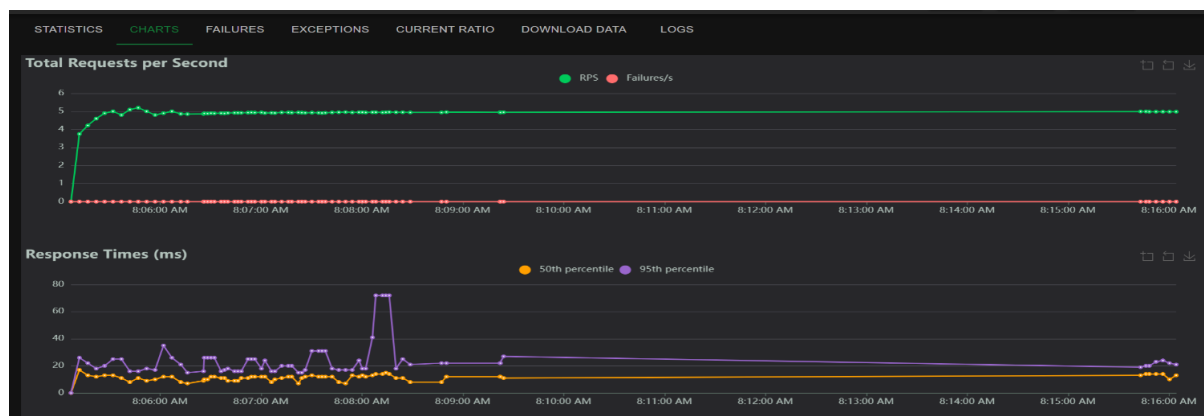
EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	265	0	7	18	30	8.19	3	77	20132	1.8	0
GET	/about	156	0	7	14	21	7.86	3	32	7936	0.5	0
POST	/predict	409	0	14	26	43	15.42	5	73	8633	2.5	0
	Aggregated	830	0	11	23	38	11.69	3	77	12173.37	4.8	0



Activity 6.2: Prepare Deployment Files

Requirements.txt, Procfile, and runtime.txt are created to support deployment on a cloud platform such as Railway.

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS						
# Failures	Method	Name	Message	First Seen	Last Seen	

Activity 6.3: Deploy to Railway and Obtain Live URL

The project is pushed to GitHub and connected to Railway, which automatically builds and deploys the application, generating a public URL accessible from anywhere.

```

locustfile.py
Procfile
requirements.txt
runtime.txt

```

Conclusion:

Rising Waters is a machine learning-powered flood prediction platform built with Flask and styled with a modern glassmorphism interface, designed to support early flood warning and disaster response decision-making. It leverages four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost — trained on a real-world dataset of over one million records to instantly classify flood risk based on twenty environmental and meteorological parameters. The XGBoost model, achieving an accuracy of 81.60%, was selected for deployment due to its superior performance among the four algorithms tested.

The project, which included dataset preparation, model training and evaluation, Flask backend development, frontend design, and performance testing, demonstrates how a structured machine learning pipeline can be transformed into a practical, user-friendly disaster management tool. Load testing using Locust confirmed that the application performs reliably under concurrent user load with zero failures and response times well within acceptable thresholds, validating its readiness for real-world use.

Looking ahead, Rising Waters offers significant opportunities for expansion. Future enhancements include integration with real-time weather APIs to automatically populate input fields with live meteorological data, implementation of SMS and email-based alert systems for automated flood warnings, addition of a prediction history dashboard to track and analyse past assessments, and multi-language support to make the system accessible to regional disaster management teams across different states. By continuously retraining the model with larger datasets and refining the user interface, the platform is well-positioned to evolve from an academic prototype into a fully operational flood early warning system supporting government agencies and disaster relief organizations.